

1. N-Queens Visualization — N = 4, 5, 8

```
main.py Visualize Run Output

1 def solve_n_queens(n):
2     board = [-1] * n
3     solutions = []
4
5     def is_safe(row, col):
6         for r in range(row):
7             if board[r] == col or abs(board[r] - col) == abs(r - row):
8                 return False
9         return True
10
11    def backtrack(row):
12        if row == n:
13            solutions.append(board.copy())
14            return
15
16        for col in range(n):
17            if is_safe(row, col):
18                board[row] = col
19                backtrack(row + 1)
20                board[row] = -1
21
22    backtrack(0)
23    return solutions
24
25
```

```
=====
N = 4
Number of solutions: 2
=====

Solution 1
. Q . .
. . . Q
Q . . .
. . Q .

Solution 2
. . Q .
Q . . .
. . . Q
. Q . .

=====
N = 5
Number of solutions: 10
=====

Solution 1
Q . . . .
```

2. Generalized N-Queens Problems

```
main.py Visualize Run Output

1 def solve_general(rows, cols, obstacles=set(), allowed=None):
2     board = [-1] * rows
3     solution = None
4
5     def is_safe(row, col):
6         # Check previous queens
7         for r in range(row):
8             c = board[r]
9
10            if c == col:
11                return False
12
13            if abs(r - row) == abs(c - col):
14                return False
15
16        return True
17
18    def backtrack(row):
19        nonlocal solution
20
21        if row == rows:
22            solution = board.copy()
23            return
24
25        for col in range(cols):
26            if is_safe(row, col):
27                board[row] = col
28                backtrack(row + 1)
29                board[row] = -1
30
31    backtrack(0)
32    return solution
33
```

```
===== 8 x 10 BOARD =====
Solution: [1, 3, 5, 2, 8, 10, 4, 7]
Q . . . . . . . .
. . Q . . . . . .
. . . Q . . . . .
. Q . . . . . . .
. . . . . Q . . .
. . . . . . Q . .
. . . Q . . . . .
. . . . . Q . . .

===== 5 x 5 BOARD WITH OBSTACLES =====
Solution: [1, 3, 5, 2, 4]
Q . . . .
. X Q . .
. . . Q .
. Q . X .
. . . Q .

===== 6 x 6 BOARD WITH RESTRICTION =====
Solution: [2, 4, 6, 1, 3, 5]
```

3. SUDOKU SOLVER

```
main.py Visualize Run Output

1 def solve(board):
2     for r in range(9):
3         for c in range(9):
4             if board[r][c] == '.':
5                 for n in '123456789':
6                     if valid(board, r, c, n):
7                         board[r][c] = n
8                         if solve(board):
9                             return True
10                        board[r][c] = '.'
11                    return False
12    return True
13
14
15 def valid(board, r, c, n):
16     for i in range(9):
17         if board[r][i] == n or board[i][c] == n:
18             return False
19
20     br = (r // 3) * 3
21     bc = (c // 3) * 3
22     for i in range(3):
23         for j in range(3):
24             if board[br+i][bc+j] == n:
25                 return False
26     return True
27
```

```
Solved Sudoku:
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9

=== Code Execution Successful ===
```

4. SUDOKU SOLVER

```
def valid(board, r, c, n):
    for i in range(br, br + 3):
        for j in range(bc, bc + 3):
            if board[i][j] == n:
                return False
    return True

board = [
    ["5","3",".", ".", ".", "7",".", ".", ".", "."],
    ["6",".", ".", ".", "1","9","5",".", ".", "."],
    [".","9","8",".", ".", ".", ".", "6",".", "."],
    ["8",".", ".", ".", "6",".", ".", ".", "3","."],
    ["4",".", ".", "8",".", "3",".", ".", ".", "1"],
    ["7",".", ".", ".", "2",".", ".", ".", "6","."],
    [".","6",".", ".", ".", ".", "2","8","."],
    [".",".", ".", "4","1","9",".", ".", "5"],
    [".",".", ".", ".", "8",".", ".", "7","9"]
]

solve(board)

print("Solved Sudoku:")
```

Solved Sudoku:

```
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9
```

=== Code Execution Successful ===

5. TARGET SUM

```
moin.py Visualize Run Output
1 def target_sum(nums, target):
2     count = 0
3
4     def backtrack(i, total):
5         nonlocal count
6
7         if i == len(nums):
8             if total == target:
9                 count += 1
10            return
11
12            backtrack(i + 1, total + nums[i])
13            backtrack(i + 1, total - nums[i])
14
15        backtrack(0, 0)
16        return count
17
18
19 nums = [1, 1, 1, 1, 1]
20 target = 3
21
22 print("Number of expressions:", target_sum(nums, target))
```

Number of expressions: 5

=== Code Execution Successful ===

6. SUM OF SUBARRAY MINIMUMS

```
def sum_subarray_mins(arr):
    MOD = 10**9 + 7
    total = 0

    for i in range(len(arr)):
        minimum = arr[i]

        for j in range(i, len(arr)):
            minimum = min(minimum, arr[j])
            total += minimum

    return total % MOD

arr = [3, 1, 2, 4]

print("Sum of subarray minimums:", sum_subarray_mins(arr))
```

Sum of subarray minimums: 17

=== Code Execution Successful ===

7. COMBINATION SUM

```
def combination_sum(candidates, target):
    result = []

    def backtrack(start, total, path):
        if total == target:
            result.append(path.copy())
            return

        if total > target:
            return

        for i in range(start, len(candidates)):
            path.append(candidates[i])
            backtrack(i, total + candidates[i], path)
            path.pop()

    backtrack(0, 0, [])
    return result

candidates = [2, 3, 6, 7]
target = 7

print("Combinations:", combination_sum(candidates, target))
```

Combinations: [[2, 2, 3], [7]]

=== Code Execution Successful ===

8. COMBINATION SUM II

```
def combination_sum2(candidates, target):
    result = []
    candidates.sort()

    def backtrack(start, total, path):
        if total == target:
            result.append(path.copy())
            return

        for i in range(start, len(candidates)):
            if i > start and candidates[i] == candidates[i - 1]:
                continue

            if total + candidates[i] > target:
                break

            path.append(candidates[i])
            backtrack(i + 1, total + candidates[i], path)
            path.pop()

    backtrack(0, 0, [])
    return result

candidates = [10, 1, 2, 7, 6, 1, 5]
```

Combinations:

[1, 1, 6]
[1, 2, 5]
[1, 7]
[2, 6]

=== Code Execution Successful ===

9. PERMUTATIONS

```
def permutations(nums):
    result = []

    def backtrack(path, used):
        if len(path) == len(nums):
            result.append(path.copy())
            return

        for i in range(len(nums)):
            if not used[i]:
                used[i] = True
                path.append(nums[i])

                backtrack(path, used)

                path.pop()
                used[i] = False

    backtrack([], [False] * len(nums))
    return result

nums = [1, 2, 3]

print("Permutations:")
```

Permutations:

[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]

=== Code Execution Successful ===

10. UNIQUE PERMUTATIONS

```
def unique_permutations(nums):
    nums.sort()
    result = []

    def backtrack(path, used):
        if len(path) == len(nums):
            result.append(path.copy())
            return

        for i in range(len(nums)):
            if used[i]:
                continue

            if i > 0 and nums[i] == nums[i - 1] and not used[i - 1]:
                continue

            used[i] = True
            path.append(nums[i])

            backtrack(path, used)

            path.pop()
            used[i] = False

    backtrack([], [False] * len(nums))
```

Unique Permutations:
[[1, 1, 2], [1, 2, 1], [2, 1, 1]]

=== Code Execution Successful ===

11. GRAPH COLORING – MINIMUM COLORS

```
def graph_coloring(edges, n):
    graph = [[] for _ in range(n)]

    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    color = [-1] * n

    def safe(v, c):
        for neighbor in graph[v]:
            if color[neighbor] == c:
                return False
        return True

    def solve(v, k):
        if v == n:
            return True

        for c in range(k):
            if safe(v, c):
                color[v] = c

                if solve(v + 1, k):
                    return True

    solve(0, 3)
```

Minimum number of colors: 3
Vertex colors: [0, 1, 2, 1]

=== Code Execution Successful ===

12. GRAPH COLORING – K = 3

```
def graph_coloring(edges, n, k):
    graph = [[] for _ in range(n)]

    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    color = [-1] * n

    def safe(v, c):
        for u in graph[v]:
            if color[u] == c:
                return False
        return True

    def solve(v):
        if v == n:
            return True

        for c in range(k):
            if safe(v, c):
                color[v] = c

                if solve(v + 1):
                    return True

    solve(0)
```

Can be colored with 3 colors: True
Vertex colors: [0, 1, 2, 1]

=== Code Execution Successful ===

13. HAMILTONIAN CYCLE

```
def hamiltonian_cycle(edges, n):
    def solve():
        for v in graph[path[-1]]:
            if v not in visited:
                visited.remove(v)

                return False

        if solve():
            return True, path + [0]

        return False, []

    edges = [
        (0, 1), (1, 2), (2, 3),
        (3, 0), (0, 2), (2, 4), (4, 0)
    ]

    n = 5

    possible, cycle = hamiltonian_cycle(edges, n)

    print("Hamiltonian Cycle Exists:", possible)
```

Hamiltonian Cycle Exists: False

=== Code Execution Successful ===

14. HAMILTONIAN CYCLE

```
def hamiltonian_cycle(edges, n):
    graph = [[] for _ in range(n)]

    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)

    path = [0]
    visited = {0}

    def solve():
        if len(path) == n:
            return path[0] in graph[path[-1]]

        for v in graph[path[-1]]:
            if v not in visited:
                visited.add(v)
                path.append(v)

                if solve():
                    return True

                path.pop()
                visited.remove(v)

    return solve()
```

Hamiltonian Cycle Exists: True
Cycle: [0, 1, 2, 3, 0]

=== Code Execution Successful ===

15. GENERATE ALL SUBSETS IN LEXICOGRAPHICAL ORDER

```
def subsets(nums):
    result = []

    def backtrack(index, path):
        if index == len(nums):
            result.append(path.copy())
            return

        # Exclude current element
        backtrack(index + 1, path)

        # Include current element
        path.append(nums[index])
        backtrack(index + 1, path)
        path.pop()

    backtrack(0, [])

    return result

S = [1, 2, 3]

result = subsets(S)
```

All subsets in lexicographical order:

[
[1]
[1, 2]
[1, 2, 3]
[1, 3]
[2]
[2, 3]
[3]

=== Code Execution Successful ===

16. SUBSET GENERATION

```
def subsets(nums, x):
    def backtrack(i, path):
        if i == len(nums):
            if x in path:
                result.append(path.copy())
            return

        # Exclude current element
        backtrack(i + 1, path)

        # Include current element
        path.append(nums[i])
        backtrack(i + 1, path)
        path.pop()

    backtrack(0, [])
    return result

E = [2, 3, 4, 5]
x = 3

result = subsets(E, x)

print("Subsets containing", x, ":")
```

Subsets containing 3 :

- [3]
- [3, 5]
- [3, 4]
- [3, 4, 5]
- [2, 3]
- [2, 3, 5]
- [2, 3, 4]
- [2, 3, 4, 5]

=== Code Execution Successful ===

17. WORD SUBSETS

```
def word_subsets(words1, words2):
    for word in words2:
        # Check each word in words1
        for word in words1:
            count = {}

            for ch in word:
                count[ch] = count.get(ch, 0) + 1

            valid = True

            for ch in required:
                if count.get(ch, 0) < required[ch]:
                    valid = False
                    break

            if valid:
                result.append(word)

    return result

words1 = ["amazon", "apple", "facebook", "google", "leetcode"]
words2 = ["facebook", "google", "leetcode"]
```

Universal words: ['facebook', 'google', 'leetcode']

=== Code Execution Successful ===